



Pixelado de Rostros de Menores

Sistema Distribuido Event-Driven · Apache Kafka · YOLOv8 · MobileNetV2

Problema y Objetivo

1 Problema real

Proteger la identidad de menores en imágenes digitales de forma automática, escalable y sin intervención humana.

2 Solución propuesta

Pipeline event-driven con Kafka: detecta caras con YOLOv8, estima la edad con MobileNetV2 y pixela automáticamente si es menor.

3 Doble modalidad

Procesamiento de fotos estáticas y detección en tiempo real a 60fps con webcam, sin almacenamiento de sesión.

¿Qué hace el sistema?

1 Detecta rostros automáticamente

El servicio de **Face Detection** analiza cada imagen recibida, localiza todos los rostros presentes y genera bounding boxes precisos usando OpenCV o YOLO.

2 Estima la edad de cada rostro detectado

El servicio de **Age Detection** aplica una red neuronal (TensorFlow / PyTorch) a cada recorte de rostro para obtener una estimación de edad en años.

3 Pixela automáticamente los rostros de menores

Cuando la edad estimada es $< 18^*$, el Pixelation Service aplica un efecto de pixelado antes de almacenar la imagen final en MinIO (S3).

Microservicios del Sistema

01

INPUT

FastAPI

API Gateway

Recibe imágenes vía HTTP/REST y publica el evento inicial en el topic images.raw.

02

CTRL

Python · Kafka Consumer

Orquestador

Coordina el flujo completo, supervisa estados y gestiona errores del pipeline.

03

DETECT

OpenCV / YOLO

Face Detection

Detecta todos los rostros en la imagen y genera bounding boxes precisos.

04

INFER

TensorFlow / PyTorch

Age Detection

Aplica una red neuronal a cada recorte de rostro y clasifica menores de edad.

05

PROCESS

OpenCV

Pixelation Service

Pixela los rostros de los menores identificados.

06

STORE

MinIO + PostgreSQL

Storage Service

Persiste imágenes en MinIO (S3) y guarda metadata en PostgreSQL.

Flujo del Pipeline

API 1	Cliente sube imagen
Face Detection	Detecta caras con YOLO → bounding boxes
02	Registra las caras en BD
Age Detection	Estima las edades de las caras
03	Actualiza edad de cada cara en BD
Pixelation	Pixela a los menores → marcos con detalle
04	Monta la imagen pixelada y con marcos → URL para API 2

Latencias del proceso

01 Detección de caras (YOLOv8)

300–800 ms

02 Estimación de edad × cara

200–600 ms

03 Pixelado + marcos

50–150 ms

04 Total end-to-end

< 2 s (1–3 caras)

Estrategia de Almacenamiento en MinIO

Bucket: minio-raw

Almacena la imagen original tal cual llega del cliente (API 1). Es la fuente de verdad para el pipeline de detección.

Bucket: minio-faces (Individuales)

Recortes de caras detectadas por YOLO. Se guardan dos versiones por cada cara: **Pixelada** (menores) o con **Marco** de detalle.

Bucket: minio-processed (Imagen Final)

Imagen completa reconstruida con las ofuscaciones aplicadas. Lista para ser servida por la API 2.

Topics de Kafka



Topic	Productor	Consumidor
<code>images.raw</code>	API Gateway	Orquestador / Face Detection
<code>images.faces_detected</code>	Face Detection	Orquestador O2
<code>images.age_estimated</code>	Age Detection	Orquestador O3
<code>images.processed</code>	Pixelation Svc	Orquestador O4 → Storage

Modelo de predicción de Edad

MobileNetV2 (fine-tuned)

Umbral conservador

22 años (margen de seguridad)

Datos de Entrenamiento

85 % entrenamiento / 15 % validación
Splits por franjas de edad

Aumento de datos

Volteo horizontal, Rotación aleatoria, ColorJitter, RandomGrayscale, RandomErasing

WeightedRandomSampler

x1.5 de 0-20 años (x2 de 13-17 años)

Arquitectura

Dropout(0.3) → Linear(1280→256) → ReLU →
Dropout(0.2) → Linear(256→1)

Entrenamiento

1 → 8 epoch - Solo cabeza (backbone congelado) - Adam, lr=1e-3
2 → 22 epoch - Backbone + cabeza (fine-tuning) - AdamW, lr diferenciado

Función de pérdida

BoundaryAwareLoss — pérdida con consciencia del umbral

$$\text{Loss} = \text{HuberLoss} + \alpha \cdot \text{BCE}$$

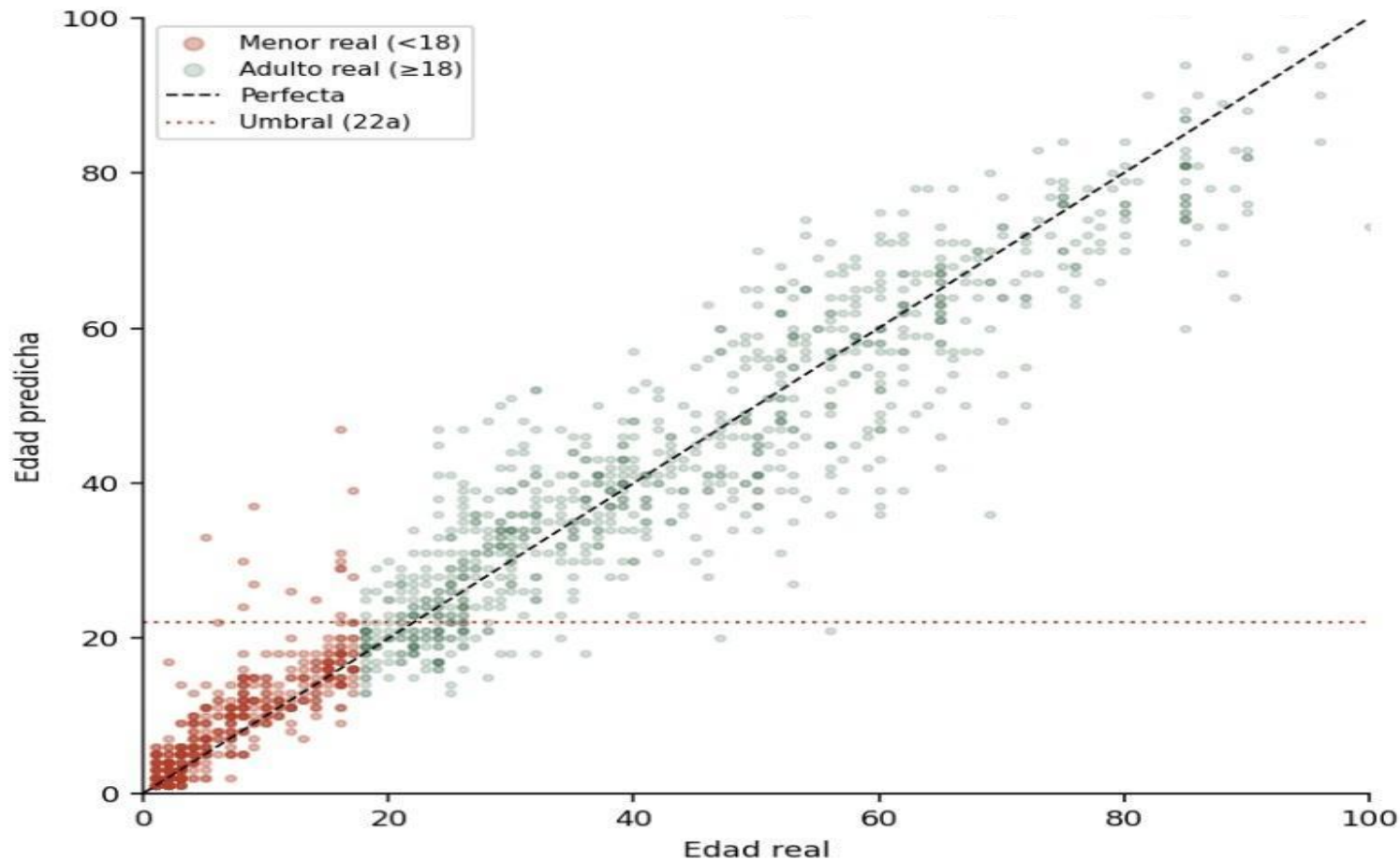
- ❑ *HuberLoss* → penaliza el error de regresión (años de diferencia), robusto a outliers
- ❑ *BCE* → penaliza cruzar el umbral de 18 años en la dirección equivocada
- ❑ $\alpha = 0.7$ → el término de clasificación pesa más que la regresión pura

Métricas del Modelo

MAE 4.5a

Recall 96.8%

Umbral 22a



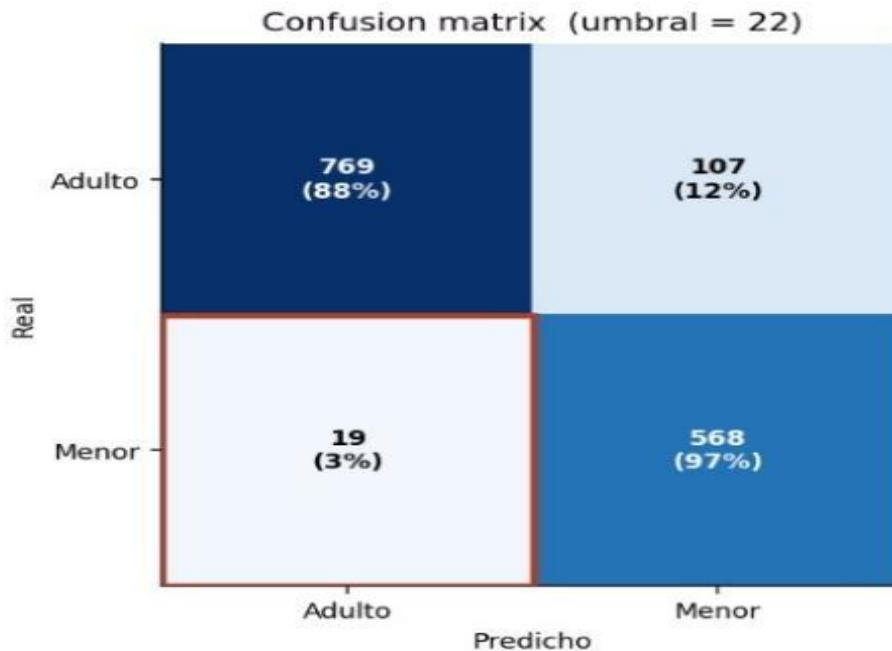
Métricas del Modelo

MAE 4.5a

Recall 96.8%

Umbral 22a

val set (1463 imgs)



Recall menores: 96.8% | FN (perdidos): 19 (3.2%) | Falsas alarmas: 12.2%

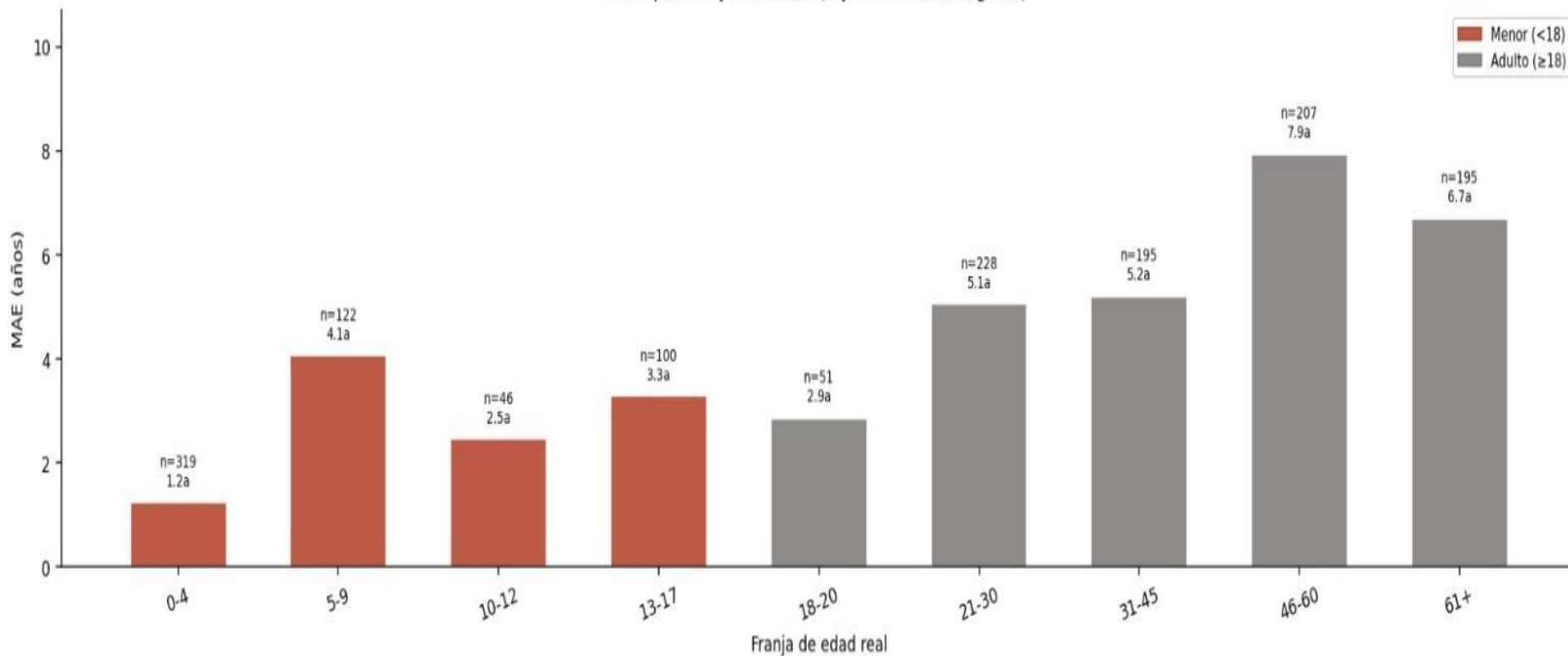
Métricas del Modelo

MAE 4.5a

Recall 96.8%

Umbral 22a

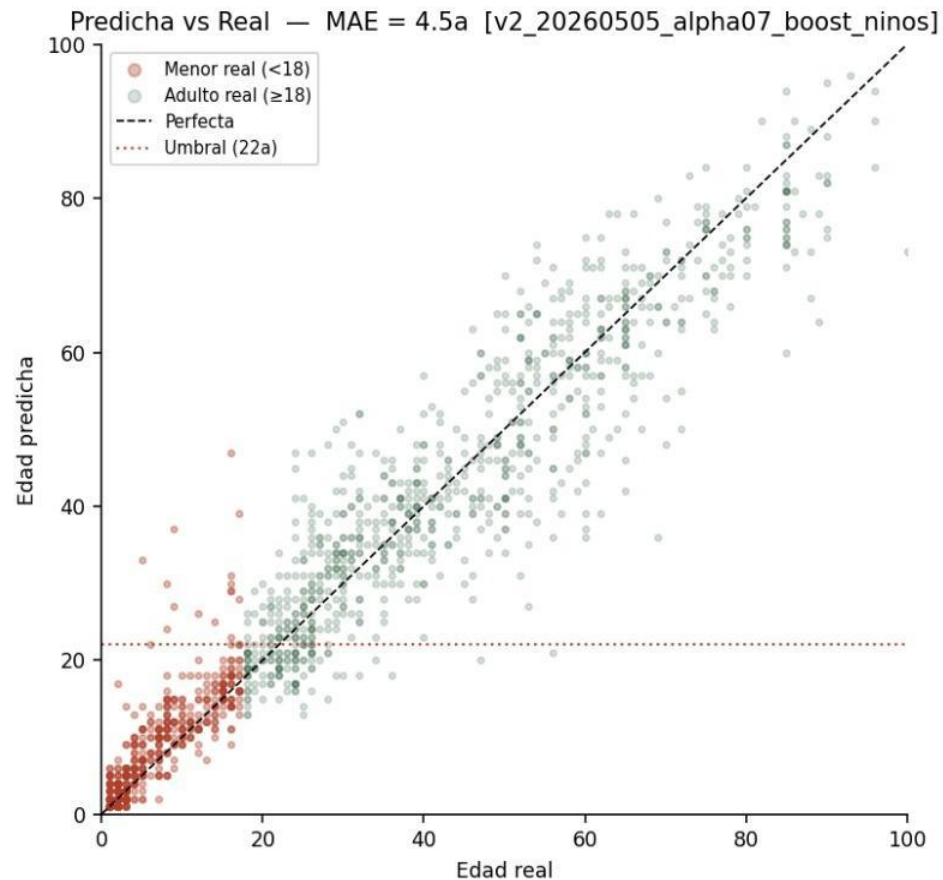
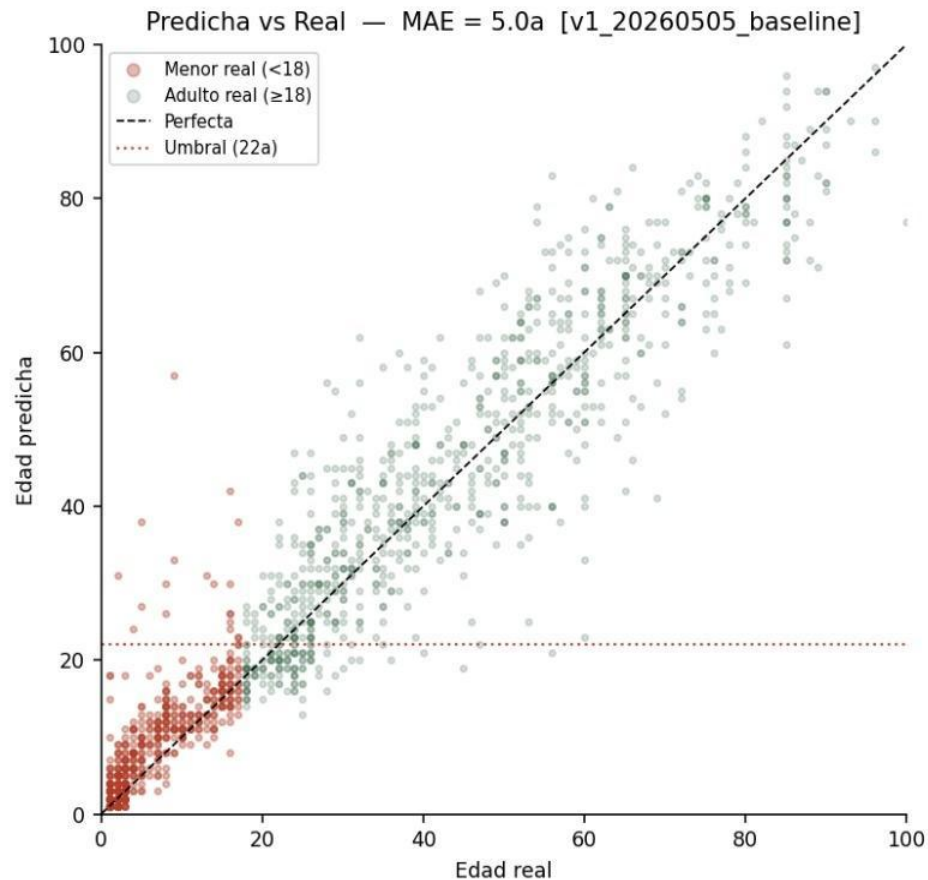
MAE por franja de edad (rojo = menores legales)



Comparativa de Modelos

v1 MAE=5.0a

v2 MAE=4.5a



Conclusiones



Sistema totalmente desacoplado

Cada microservicio evoluciona, escala y despliega de forma completamente independiente sin afectar al resto.



Alta resiliencia y tolerancia a fallos

Los fallos quedan aislados por servicio. Kafka mantiene los mensajes y permite reintentos y dead-letter queues.



Trazabilidad completa del pipeline

Timestamps en cada fase. El GUID permite reconstruir el historial completo de procesamiento de cualquier imagen.



Escalabilidad horizontal

Kafka permite múltiples instancias consumidoras en paralelo. Cada servicio escala de forma independiente según su carga.

Propuestas de Mejora

01 Modelos más precisos

Integrar modelos transformer para estimación de edad con mayor precisión y menor sesgo.

02 Panel de monitorización

Dashboard en tiempo real con Kafka UI + Grafana para observar el estado del pipeline.

03 Soporte para vídeo

Procesar vídeo frame a frame, no solo imágenes, con gestión de offsets temporales.

04 Autenticación y auditoría

Control de acceso RBAC con logs de auditoría completos por usuario y solicitud.

05 API de notificaciones

Webhooks para notificar al cliente cuando el procesamiento de su imagen finalice.

06 Despliegue en Kubernetes

Orquestación con K8s para auto-scaling de cada microservicio según la carga.